

EXPERIMENT NO.: 7		
Title of Experiment: Implement a Simple linear regression model using python to predict a response using a single feature.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Objective

Implement a Simple Linear Regression model in Python to predict a continuous response using **one feature**.

Theory

We model the relationship between a feature xxx and target yyy as:

$$\hat{y} = b_0 + b_1 x \quad \text{or} \quad y = b_0 + b_1 x$$

where b_0 is the intercept and b_1 is the slope learned by minimizing Mean Squared Error.

Dataset

We'll use scikit-learn's **Diabetes** dataset (built-in, no download needed). We'll predict the disease progression score (target) from the **BMI** feature (bmi) only.

Step-by-step Procedure

1. Import libraries

- o `numpy, matplotlib` for math/plots
- o `sklearn.datasets` to load data
- o `sklearn.model_selection` for train/test split
- o `sklearn.linear_model` for LinearRegression
- o `sklearn.metrics` for evaluation

2. Load dataset & choose one feature

- o Load with `load_diabetes()`
- o Select the single column `bmi` as XXX (keep it 2D: `shape (n_samples, 1)`)

- o Use `target` as `yyy`
- 3. **Split data**
 - o Train/Test split (e.g., 80/20) with a fixed `random_state` for reproducibility.
- 4. **Train the model**
 - o Fit `LinearRegression()` on training data.
- 5. **Inspect learned parameters**
 - o Print slope `b1b_1b1` and intercept `b0b_0b0`.
- 6. **Predict on test data**
 - o Use `model.predict(X_test)` to get $\hat{y}$.
- 7. **Evaluate**
 - o Compute **MAE**, **MSE**, **RMSE**, **R²**.
- 8. **Visualize**
 - o Scatter plot of training and test points.
 - o Plot the regression line $\hat{y} = b_0 + b_1 x$ across the feature range.
- 9. **(Optional) Save model**
 - o Use `joblib` or `pickle` if you want to persist the model.

Simple Linear Regression with a Single Feature (BMI) on the Diabetes dataset

===== 1) Imports =====

import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

===== 2) Load data & select single feature =====

diabetes = load_diabetes()

X_all = diabetes.data # standardized features

```

y = diabetes.target          # disease progression
feature_names = diabetes.feature_names

# Choose single feature: 'bmi'
bmi_idx = feature_names.index('bmi')
X = X_all[:, [bmi_idx]]      # keep 2D shape: (n_samples, 1)

print("Feature chosen:", feature_names[bmi_idx])
print("X shape:", X.shape, " y shape:", y.shape)

```

Output -

```

PS C:\Users\sarve\Desktop\Python> p
Feature chosen: bmi
X shape: (442, 1) y shape: (442,)

```

```

# ===== 3) Train/Test split =====
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

# ===== 4) Train model =====
model = LinearRegression()
model.fit(X_train, y_train)

```

```

# ===== 5) Parameters =====
b0 = model.intercept_
b1 = model.coef_[0]
print(f"Intercept (b0): {b0:.4f}")
print(f"Slope (b1) for BMI: {b1:.4f}")

```

Output -

```
Intercept (b0): 152.0034  
Slope (b1) for BMI: 998.5777
```

```
# ===== 6) Predict on test =====
```

```
y_pred = model.predict(X_test)
```

```
# ===== 7) Evaluate =====
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nEvaluation on Test Set")
```

```
print(f"MAE : {mae:.3f}")
```

```
print(f"MSE : {mse:.3f}")
```

```
print(f"RMSE : {rmse:.3f}")
```

```
print(f"R^2 : {r2:.3f}")
```

Output -

```
Evaluation on Test Set  
MAE   : 52.260  
MSE   : 4061.826  
RMSE  : 63.732  
R^2   : 0.233
```

```
# ===== 8) Visualize =====
```

```
# Make a smooth line across the feature range
```

```
x_min, x_max = X.min(), X.max()
```

```
x_line = np.linspace(x_min, x_max, 200).reshape(-1, 1)
```

```
y_line = model.predict(x_line)
```

```
plt.figure(figsize=(7, 5))
```

```
# training points
```

```
plt.scatter(X_train, y_train, alpha=0.7, label="Train")
```

```
# test points
```

```
plt.scatter(X_test, y_test, alpha=0.9, marker='x', label="Test")
```

```
# regression line
```

```
plt.plot(x_line, y_line, linewidth=2, label="Regression line")
```

```
plt.title("Simple Linear Regression: BMI → Disease Progression")
```

```
plt.xlabel("BMI (standardized)")
```

```
plt.ylabel("Disease progression")
```

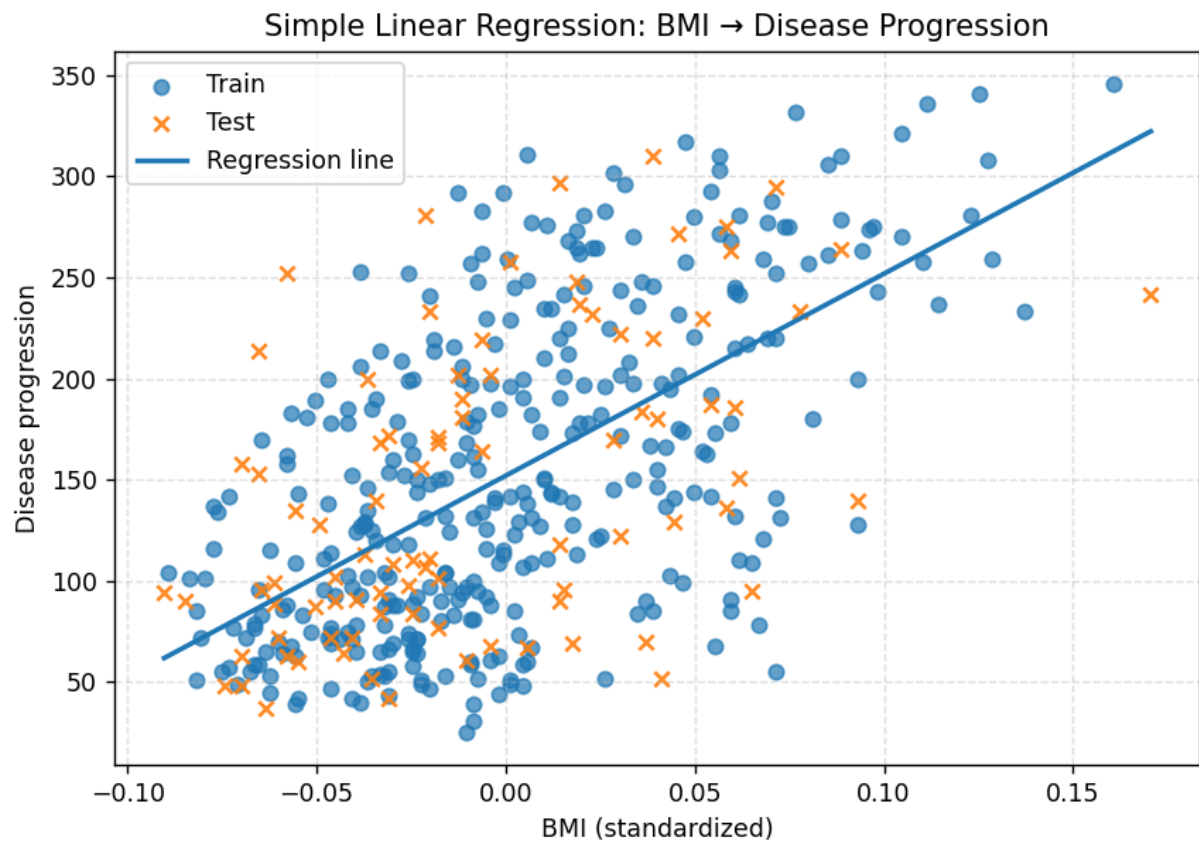
```
plt.legend()
```

```
plt.grid(True, linestyle="--", alpha=0.4)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output -



===== 9) Quick usage example =====

Predict for a specific BMI value (standardized scale)

example_bmi = 0.05 # small positive standardized BMI

pred = model.predict(np.array([[example_bmi]]))[0]

print(f"\nPredicted progression for BMI={example_bmi:.2f}: {pred:.2f}")

Output -

Predicted progression for BMI=0.05: 201.93

EXPERIMENT NO.: 8		
Title of Experiment: Implement a Multiple linear regression model to show the relationship between two or more features and a response by fitting a linear equation to observed data.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Objective: Fit a linear equation relating two or more input features to a continuous response and evaluate the quality of the model.

Step-by-step solution (concise)

1. **Aim** — Build a multiple linear regression model to predict a continuous target from multiple input features.
2. **Theory (brief)** — Model:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_p x_p$$
Parameters b_j are found by minimizing the residual sum of squares (ordinary least squares). Evaluate with MSE/RMSE/MAE/ R^2 and check assumptions (linearity, independence, homoscedasticity, normality of residuals, multicollinearity).
3. **Libraries / tools required** — `numpy`, `pandas`, `matplotlib`, `scikit-learn`. (Optional: `statsmodels` for summary; not required.)
4. **Dataset** — use `scikit-learn`'s `load_diabetes()` (built-in). We'll:
 - o Demonstrate **Model A**: two features (`bmi`, `bp`) — meets "two or more features".
 - o Demonstrate **Model B**: all available features — shows multi-feature behaviour.
5. **Preprocessing**
 - o Put data into a `DataFrame`.
 - o Check for missing values / outliers.
 - o Train/test split (e.g., 80/20, `random_state=42`).
 - o Scale features (recommended for comparability and numeric stability).
6. **Training**
 - o Fit `LinearRegression()` on training set.
7. **Parameters & interpretation**
 - o Print intercept and coefficients. Map coefficients to feature names: coefficient tells change in response per unit change in feature (holding others constant).
8. **Evaluation**
 - o Compute MAE, MSE, RMSE, R^2 and **Adjusted R^2** (adjusted for number of predictors).
 - o Compare Model A vs Model B.
9. **Diagnostics**

- o Plot predicted vs actual.
- o Plot residuals vs predicted (look for non-random patterns).
- o Compute **VIF** (variance inflation factor) to detect multicollinearity.

10. **Save / predict new values**

- o Optionally serialize model with `joblib`.
- o Demonstrate prediction on new samples (remember to scale features the same way).

"""

Multiple Linear Regression Example

- Dataset: `sklearn.datasets.load_diabetes`
- Model A: two features -> 'bmi' and 'bp'
- Model B: all features
- Outputs: coefficients, metrics (MAE, MSE, RMSE, R2, Adjusted R2), residual plots, VIF

"""

===== 1) Imports =====

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import joblib

import warnings

warnings.filterwarnings("ignore")


```
# ===== 2) Load data & build DataFrame =====
```

```
diabetes = load_diabetes()
```

```
X_all = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
```

```
y = pd.Series(diabetes.target, name='target')
```

```
print("Dataset shape:", X_all.shape)
```

```
print("Feature names:", diabetes.feature_names)
```

```
print()
```

Output -

```
Dataset shape: (442, 10)
Feature names: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
```

```
# ===== 3) Quick EDA: missing values, basic stats =====
```

```
print("Missing values per column:\n", X_all.isnull().sum())
```

```
print("\nTarget summary:\n", y.describe())
```

```
print()
```

Output -

```
Missing values per column:
age      0
sex      0
bmi      0
bp       0
s1       0
s2       0
s3       0
s4       0
s5       0
s6       0
dtype: int64

Target summary:
count    442.000000
mean     152.133484
std       77.093005
min       25.000000
25%       87.000000
50%      140.500000
75%      211.500000
max       346.000000
Name: target, dtype: float64
```

```

Missing values per column:
age      0
sex      0
bmi      0
bp       0
s1       0
s2       0
s3       0
s4       0
s5       0
s6       0
dtype: int64

Target summary:
count    442.000000
mean     152.133484
std       77.093005
min       25.000000
25%       87.000000
50%      140.500000
75%      211.500000
max      346.000000
Name: target, dtype: float64

```

===== 4) Define feature sets =====

```
features_two = ['bmi', 'bp']      # Model A (two features)
```

```
features_all = list(X_all.columns) # Model B (all features)
```

===== Helper: train, scale, fit, evaluate function =====

```
def fit_and_evaluate(feature_list, X_df, y, random_state=42):
```

```
    X = X_df[feature_list].values
```

```
    # 1) Train/test split
```

```
    X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y.values, test_size=0.2, random_state=random_state
```

```
)
```

```
    # 2) Scale features (fit on train)
```

```
    scaler = StandardScaler()
```

```
    X_train_scaled = scaler.fit_transform(X_train)
```

```
    X_test_scaled = scaler.transform(X_test)
```

```
    # 3) Fit linear regression
```

```
    model = LinearRegression()
```

```
    model.fit(X_train_scaled, y_train)
```

4) Predictions

```
y_pred_train = model.predict(X_train_scaled)
```

```
y_pred_test = model.predict(X_test_scaled)
```

5) Metrics

```
def metrics(y_true, y_pred, p):
```

```
    mae = mean_absolute_error(y_true, y_pred)
```

```
    mse = mean_squared_error(y_true, y_pred)
```

```
    rmse = np.sqrt(mse)
```

```
    r2 = r2_score(y_true, y_pred)
```

```
    n = y_true.shape[0]
```

```
    # adjusted R^2
```

```
    if n - p - 1 > 0:
```

```
        adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
```

```
    else:
```

```
        adj_r2 = np.nan
```

```
    return {'MAE': mae, 'MSE': mse, 'RMSE': rmse, 'R2': r2, 'Adj_R2': adj_r2}
```

```
metrics_train = metrics(y_train, y_pred_train, p=X_train.shape[1])
```

```
metrics_test = metrics(y_test, y_pred_test, p=X_test.shape[1])
```

6) Coefficients (map to features)

```
coef = model.coef_
```

```
intercept = model.intercept_
```

```
coef_map = dict(zip(feature_list, coef))
```

7) Residuals on test

```
residuals = y_test - y_pred_test
```

```
# 8) Compute VIF (using linear regression approach)
```

```
def compute_vif(X_mat, feature_names):
```

```
    from sklearn.linear_model import LinearRegression
```

```
    vif_values = []
```

```
    X_np = np.array(X_mat)
```

```
    p = X_np.shape[1]
```

```
    for i in range(p):
```

```
        X_i = X_np[:, i]
```

```
        X_other = np.delete(X_np, i, axis=1)
```

```
        # if only one variable left, handle separately
```

```
        if X_other.shape[1] == 0:
```

```
            r2 = 0.0
```

```
        else:
```

```
            lr = LinearRegression().fit(X_other, X_i)
```

```
            r2 = lr.score(X_other, X_i)
```

```
            vif = 1.0 / (1.0 - r2) if (1.0 - r2) > 1e-10 else np.inf
```

```
            vif_values.append(vif)
```

```
    return pd.DataFrame({'feature': feature_names, 'VIF': np.round(vif_values, 3)})
```

```
vif_df = compute_vif(X_train_scaled, feature_list)
```

```
# Pack results
```

```
results = {
```

```
    'model': model,
```

```
    'scaler': scaler,
```

```
    'features': feature_list,
```

```

    'coef_map': coef_map,
    'intercept': intercept,
    'metrics_train': metrics_train,
    'metrics_test': metrics_test,
    'X_train_scaled': X_train_scaled,
    'X_test_scaled': X_test_scaled,
    'y_train': y_train,
    'y_test': y_test,
    'y_pred_test': y_pred_test,
    'residuals': residuals,
    'vif': vif_df
}
return results

```

```

# ===== 5) Fit Model A (two features) =====
print("Fitting Model A (two features: bmi, bp)...")
res_a = fit_and_evaluate(features_two, X_all, y)
print("Intercept:", round(res_a['intercept'], 4))
print("Coefficients:")
for f, c in res_a['coef_map'].items():
    print(f" {f}: {c:.4f}")
print("Train metrics:", res_a['metrics_train'])
print("Test metrics :", res_a['metrics_test'])
print("VIF:\n", res_a['vif'])
print()

```

Output -

```

Fitting Model A (two features: bmi, bp)...
Intercept: 153.7365
Coefficients:
  bmi: 39.6323
  bp: 19.0430
Train metrics: {'MAE': 48.959273255958955, 'MSE': 3547.8578537412154, 'RMSE': 59.563897234324884,
'R2': 0.4161248413682216, 'Adj_R2': 0.41278841189032567}
Test metrics : {'MAE': 50.04411487419211, 'MSE': 3740.507019920472, 'RMSE': 61.159684596312886, '
R2': 0.2939977566766081, 'Adj_R2': 0.27757909985513385}
VIF:
  feature    VIF
0     bmi  1.184
1      bp  1.184

```

```

# ===== 6) Fit Model B (all features) =====

print("Fitting Model B (all features)...")

res_b = fit_and_evaluate(features_all, X_all, y)

print("Intercept:", round(res_b['intercept'], 4))

print("Coefficients (first 8 shown):")

for f, c in list(res_b['coef_map'].items())[:8]:
    print(f" {f}: {c:.4f}")

print("Train metrics:", res_b['metrics_train'])

print("Test metrics :", res_b['metrics_test'])

print("VIF (first 10 rows):\n", res_b['vif'].head(10))

print()

```

Output -

```

Fitting Model B (all features)...
Intercept: 153.7365
Coefficients (first 8 shown):
  age: 1.7538
  sex: -11.5118
  bmi: 25.6071
  bp: 16.8289
  s1: -44.4489
  s2: 24.6410
  s3: 7.6770
  s4: 13.1388
Train metrics: {'MAE': 43.483503523980396, 'MSE': 2868.5497028355776, 'RMSE': 53.55884336723094,
'R2': 0.5279193863361498, 'Adj_R2': 0.5141158596208325}
Test metrics : {'MAE': 42.794094679599944, 'MSE': 2900.1936284934827, 'RMSE': 53.85344583676594,
'R2': 0.45260276297191926, 'Adj_R2': 0.3824236300196012}
VIF (first 10 rows):
  feature    VIF
0    age    1.217
1    sex    1.269
2    bmi    1.574
3     bp    1.422
4     s1   55.252
5     s2   35.759
6     s3   14.286
7     s4    9.333
8     s5   10.071
9     s6    1.501

```

```

# ===== 7) Compare Model A vs Model B (test R^2 / Adj R^2) =====

print("Comparison on TEST set:")

print("Model A (features={}): R2={:.4f}, AdjR2={:.4f}".format(
    res_a['features'], res_a['metrics_test']['R2'], res_a['metrics_test']['Adj_R2']
))

print("Model B (features=all): R2={:.4f}, AdjR2={:.4f}".format(
    res_b['metrics_test']['R2'], res_b['metrics_test']['Adj_R2']
))

print()

```

Output -

```

Comparison on TEST set:
Model A (features=['bmi', 'bp']): R2=0.2940, AdjR2=0.2776
Model B (features=all): R2=0.4526, AdjR2=0.3824

```

```
# ===== 8) Diagnostic plots for Model B (all features) =====
```

```
plt.figure(figsize=(12,4))
```

```
plt.subplot(1,2,1)
```

```
plt.scatter(res_b['y_test'], res_b['y_pred_test'], alpha=0.7)
```

```
minv = min(res_b['y_test'].min(), res_b['y_pred_test'].min())
```

```
maxv = max(res_b['y_test'].max(), res_b['y_pred_test'].max())
```

```
plt.plot([minv, maxv], [minv, maxv], linestyle='--', linewidth=1, label='ideal')
```

```
plt.xlabel('Actual target (test)')
```

```
plt.ylabel('Predicted target (test)')
```

```
plt.title('Actual vs Predicted (Model B)')
```

```
plt.legend()
```

```
plt.grid(alpha=0.3)
```

```
plt.subplot(1,2,2)
```

```
plt.scatter(res_b['y_pred_test'], res_b['residuals'], alpha=0.7)
```

```
plt.axhline(0, color='black', linestyle='--')
```

```
plt.xlabel('Predicted (test)')
```

```
plt.ylabel('Residuals (test)')
```

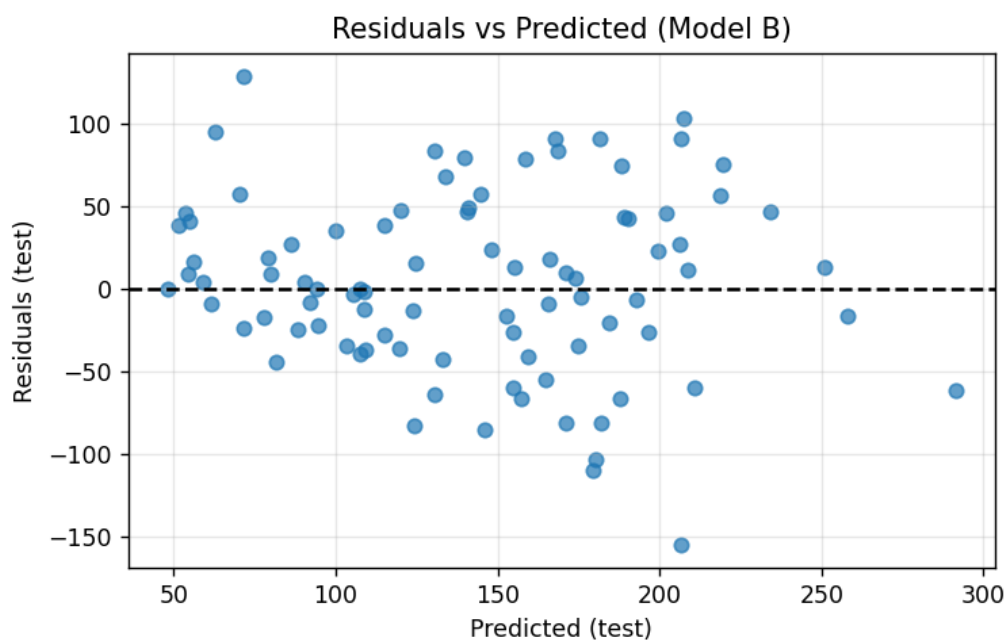
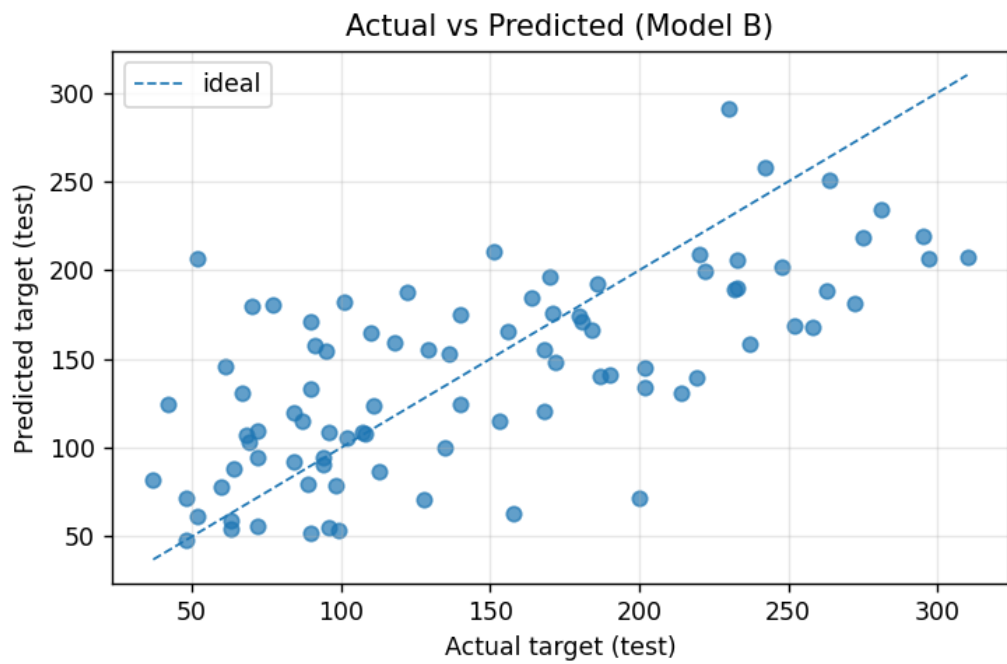
```
plt.title('Residuals vs Predicted (Model B)')
```

```
plt.grid(alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

Output -



Residual histogram / normality hint

```
plt.figure(figsize=(6,4))
```

```
plt.hist(res_b['residuals'], bins=20, edgecolor='k', alpha=0.7)
```

```
plt.title('Residuals distribution (Model B test set)')
```

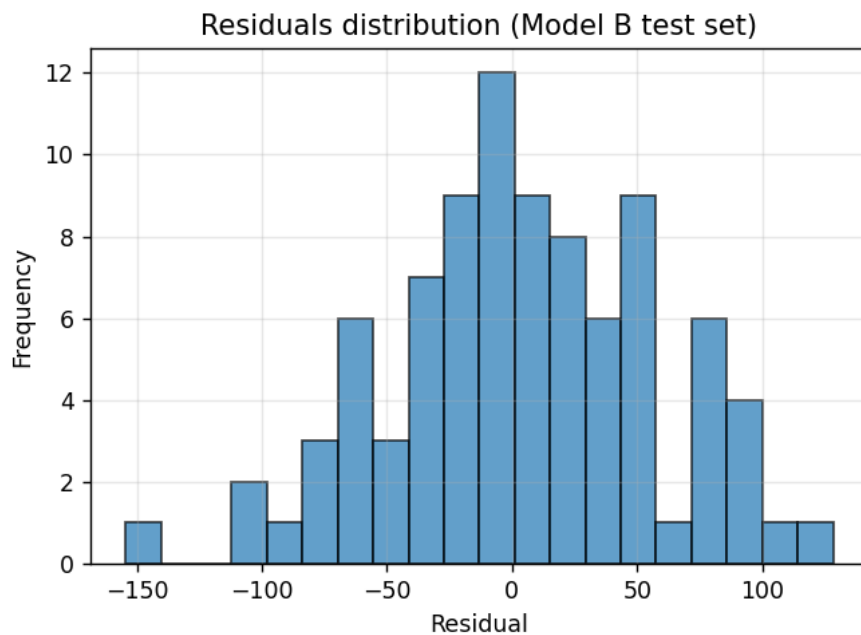
```
plt.xlabel('Residual')
```

```
plt.ylabel('Frequency')
```

```
plt.grid(alpha=0.3)
```

```
plt.show()
```

Output -



===== 9) Optional: statsmodels OLS summary (if installed) =====

try:

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(res_b['X_train_scaled'])
ols_mod = sm.OLS(res_b['y_train'], X_train_sm).fit()
print("Statsmodels OLS summary (Model B, train):")
print(ols_mod.summary())
```

except Exception as e:

```
print("statsmodels not available or failed; skipping OLS summary.")
```

Output -

```

Statsmodels OLS summary (Model B, train):
              OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.528
Model:                  OLS    Adj. R-squared:            0.514
Method:                 Least Squares    F-statistic:        38.25
Date:                   Thu, 30 Oct 2025    Prob (F-statistic):    5.41e-50
Time:                   08:13:41    Log-Likelihood:        -1906.1
No. Observations:       353    AIC:                  3834.
Df Residuals:           342    BIC:                  3877.
Df Model:                10
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	153.7365	2.896	53.083	0.000	148.040	159.433
x1	1.7538	3.195	0.549	0.583	-4.531	8.038
x2	-11.5118	3.262	-3.529	0.000	-17.929	-5.095
x3	25.6071	3.633	7.049	0.000	18.461	32.753
x4	16.8289	3.454	4.873	0.000	10.036	23.622
x5	-44.4489	21.527	-2.065	0.040	-86.792	-2.106
x6	24.6410	17.319	1.423	0.156	-9.423	58.705
x7	7.6770	10.946	0.701	0.484	-13.854	29.208
x8	13.1388	8.848	1.485	0.138	-4.264	30.542
x9	35.1612	9.191	3.826	0.000	17.083	53.239
x10	2.3514	3.548	0.663	0.508	-4.627	9.330

```

=====
Omnibus:                1.457    Durbin-Watson:          1.794
Prob(Omnibus):           0.483    Jarque-Bera (JB):        1.412
Skew:                    0.064    Prob(JB):                0.494
Kurtosis:                2.718    Cond. No.:               20.7
=====

```

===== 10) Save model B to disk =====

```

joblib.dump({'model': res_b['model'], 'scaler': res_b['scaler'], 'features':
res_b['features']},

            'linear_model_b_all_features.joblib')

print("Saved Model B to 'linear_model_b_all_features.joblib'")

```

Output -

```

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Saved Model B to 'linear_model_b_all_features.joblib'

```

===== 11) Example: predict on new samples (must scale with saved scaler) =====

```
# Build a new example using original (unscaled) feature order
example_raw = np.array([X_all.iloc[0].values]) # reuse first sample as example
# Scale using the model's scaler
scaled_example = res_b['scaler'].transform(example_raw)
pred_example = res_b['model'].predict(scaled_example)[0]
print("\nExample prediction (using first dataset row):")
print("Actual target:", float(y.iloc[0]))
print("Predicted target:", float(pred_example))
```

Output -

```
Example prediction (using first dataset row):
Example prediction (using first dataset row):
Example prediction (using first dataset row):
Actual target: 151.0
Predicted target: 210.74209144739243
```

```
# ===== 12) Short conclusion printout =====
```

```
print("\nConclusion:")
print("- Model A (2 features) vs Model B (all features) comparison above shows how adding features")
print(" can increase R^2 but may introduce multicollinearity (check VIF).")
print("- Check residual plots for non-random patterns -> indicates assumption violations.")
print("- Use VIF to detect multicollinearity; consider dropping or regularizing if VIF >> 10.")
```

Output -

```
Conclusion:
- Model A (2 features) vs Model B (all features) comparison above shows how adding features can increase R^2 but may introduce multicollinearity (check VIF).
- Check residual plots for non-random patterns -> indicates assumption violations.
- Use VIF to detect multicollinearity; consider dropping or regularizing if VIF >> 10.
```

EXPERIMENT NO.: 9		
Title of Experiment: Implement a regression model to predict house prices.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

Step-by-step solution

1. **Aim**
 - o Predict median house value using multiple input features and compare different regression models.
2. **Theory (brief)**
 - o Multiple linear regression: $\hat{y} = b_0 + b_1 x_1 + \dots + b_p x_p$
 - o Regularization (Ridge/Lasso) reduces overfitting by penalizing large coefficients.
 - o Tree-based methods (Random Forest) can capture non-linear relationships and interactions without scaling.
3. **Libraries**
 - o `numpy`, `pandas`, `matplotlib` for data & plotting.
 - o `sklearn` for dataset, models, preprocessing, metrics, cross-validation.
 - o `joblib` for saving the trained model.
4. **Load dataset**
 - o Use `fetch_california_housing()` from `sklearn.datasets`. Create `x` (features DataFrame) and `y` (target Series).
5. **Exploratory Data Analysis (EDA)**
 - o Inspect shape, feature names, head, description.
 - o Check missing values (California dataset has none).
 - o Quick correlation check and summary statistics.
6. **Train/test split**
 - o Split into training and test sets (e.g., 80/20) with `random_state` for reproducibility.
7. **Preprocessing**
 - o For linear models, scale features (StandardScaler) fitted on training set only.
 - o For Random Forest scaling is optional (doesn't change tree splits), but using scaled features is fine.
8. **Baseline model**
 - o Fit a plain `LinearRegression` to get a baseline for R^2 , RMSE.
9. **Regularized linear model (Ridge)**
 - o Use `GridSearchCV` on a small alpha grid (e.g., [0.1, 1, 10]) with 5-fold CV to find a good regularization strength.
10. **Tree based model (Random Forest)**

- o Use `GridSearchCV` (small grid) to tune `n_estimators` and `max_depth`. Tree models often perform better for this task.
- 11. **Evaluation metrics**
 - o MAE, MSE, RMSE, R^2 , **Adjusted R^2** .
 - o Also compute cross-validated RMSE on the training set to gauge generalization.
- 12. **Diagnostics**
 - o Plot Actual vs Predicted on test set.
 - o Plot residuals (residuals vs predicted) and residual histogram.
 - o Feature importance from Random Forest to interpret important predictors.
- 13. **Select best model**
 - o Compare test RMSE / R^2 and choose the model with best generalization (usually lower RMSE & higher R^2).
- 14. **Save best model**
 - o Persist with `joblib.dump()` so it can be loaded later.
- 15. **Report**
 - o Record coefficients (for linear models), best hyperparameters, test metrics, diagnostic plots, and a short conclusion.

"""

House Price Regression Example (California Housing)

- Baseline: `LinearRegression`
- Regularized: `Ridge` (`GridSearchCV`)
- Tree-based: `RandomForestRegressor` (`GridSearchCV`)
- Outputs: metrics, diagnostic plots, feature importance, save best model

"""

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
# ===== 1) Imports =====
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import joblib
```

```
from sklearn.datasets import fetch_california_housing
```

```
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LinearRegression, Ridge
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# ===== 2) Load dataset =====
```

```
data = fetch_california_housing()
```

```
X = pd.DataFrame(data.data, columns=data.feature_names)
```

```
y = pd.Series(data.target, name='MedHouseVal') # target units ~ (100k USD)
```

```
print("Dataset shape:", X.shape)
```

```
print("Feature names:", list(X.columns))
```

```
print()
```

Output -

```
Dataset shape: (20640, 8)
Feature names: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',
'Population', 'AveOccup', 'Latitude', 'Longitude']
```

```
# ===== 3) Quick EDA =====
```

```
print("First 5 rows:\n", X.head())
```

```
First 5 rows:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude
0  8.3252    41.0    6.984127   1.023810     322.0    2.555556    37.88    -122.23
1  8.3014    21.0    6.238137   0.971880     2401.0    2.109842    37.86    -122.22
2  7.2574    52.0    8.288136   1.073446     496.0    2.802260    37.85    -122.24
3  5.6431    52.0    5.817352   1.073059     558.0    2.547945    37.85    -122.25
4  3.8462    52.0    6.281853   1.081081     565.0    2.181467    37.85    -122.25
```

```
print("\nTarget summary:\n", y.describe())
```

```
Target summary:
count      20640.000000
mean        2.068558
std         1.153956
min         0.149990
25%         1.196000
50%         1.797000
75%         2.647250
max         5.000010
Name: MedHouseVal, dtype: float64
```

```
print("\nMissing values per column:\n", X.isnull().sum())
print()
```

```
Missing values per column:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
dtype: int64
```

```
# Optional: show correlation with target
corr_with_target = X.join(y).corr()['MedHouseVal'].sort_values(ascending=False)
print("Top correlations with target:\n", corr_with_target.head(10))
print()
```



```

Top correlations with target:
MedHouseVal    1.000000
MedInc         0.688075
AveRooms       0.151948
HouseAge       0.105623
AveOccup       -0.023737
Population     -0.024650
Longitude      -0.045967
AveBedrms      -0.046701
Latitude       -0.144160
Name: MedHouseVal, dtype: float64

```

===== 4) Train/Test split =====

RANDOM_STATE = 42

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)
print("Train shape:", X_train.shape, " Test shape:", X_test.shape)
print()

```

```

Train shape: (16512, 8)  Test shape: (4128, 8)

```

===== 5) Scale features (fit scaler on train only) =====

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

helper: adjusted R²

```

def adjusted_r2(r2, n, p):
    if n - p - 1 <= 0:
        return np.nan
    return 1 - (1 - r2) * (n - 1) / (n - p - 1)

```

```

# helper: evaluation report
def evaluate_model(name, model, X_tr, X_te, y_tr, y_te):
    y_pred = model.predict(X_te)
    y_pred_train = model.predict(X_tr)
    n_test = y_te.shape[0]
    p = X_tr.shape[1]
    mae = mean_absolute_error(y_te, y_pred)
    mse = mean_squared_error(y_te, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_te, y_pred)
    adj = adjusted_r2(r2, n_test, p)
    # cross-validated RMSE on train (5-fold)
    cv_scores = cross_val_score(model, X_tr, y_tr, scoring='neg_mean_squared_error',
                                cv=5, n_jobs=-1)
    cv_rmse = np.sqrt(-cv_scores).mean()
    print(f"---- {name} ----")
    print(f"MAE: {mae:.4f}")
    print(f"MSE: {mse:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"R2: {r2:.4f}")
    print(f"AdjR2:{adj:.4f}")
    print(f"Train CV RMSE (5-fold): {cv_rmse:.4f}")
    print()

```

```

---- LinearRegression ----
MAE:  0.5332
MSE:  0.5559
RMSE: 0.7456
R2:   0.5758
AdjR2:0.5750
Train CV RMSE (5-fold): 0.7205

```

```

    return {'name': name, 'mae': mae, 'mse': mse, 'rmse': rmse, 'r2': r2, 'adj_r2': adj,
            'cv_rmse': cv_rmse,
            'y_pred': y_pred}

```

```

# ===== 6) Baseline: Linear Regression =====

```

```

lin = LinearRegression()

```

```

lin.fit(X_train_scaled, y_train)

```

```

res_lin = evaluate_model("LinearRegression", lin, X_train_scaled, X_test_scaled,
y_train, y_test)

```

```

# show coefficients mapped to features

```

```

coef_map = pd.Series(lin.coef_, index=X.columns).sort_values(key=abs,
ascending=False)

```

```

print("Linear Regression coefficients (by absolute value):\n", coef_map)

```

```

print()

```

```

Linear Regression coefficients (by absolute value):
Latitude      -0.896929
Longitude     -0.869842
MedInc         0.854383
AveBedrms      0.339259
AveRooms       -0.294410
HouseAge       0.122546
AveOccup       -0.040829
Population     -0.002308
dtype: float64

```

```

# ===== 7) Ridge Regression (GridSearchCV) =====

```

```

ridge = Ridge(random_state=RANDOM_STATE, max_iter=5000)

```

```

param_grid_ridge = {'alpha': [0.1, 1.0, 10.0, 50.0]}

```

```

gs_ridge = GridSearchCV(ridge, param_grid_ridge,
scoring='neg_mean_squared_error', cv=5, n_jobs=-1)

```

```

gs_ridge.fit(X_train_scaled, y_train)

```

```

best_ridge = gs_ridge.best_estimator_
print("Ridge best params:", gs_ridge.best_params_)
res_ridge = evaluate_model("Ridge (best)", best_ridge, X_train_scaled, X_test_scaled,
y_train, y_test)
print("Ridge coefficients:\n", pd.Series(best_ridge.coef_,
index=X.columns).sort_values(key=abs, ascending=False))
print()

```

```

Ridge best params: {'alpha': 0.1}
---- Ridge (best) ----

MAE:  0.5332
MSE:  0.5559
RMSE: 0.7456
R2:   0.5758
AdjR2:0.5750
Train CV RMSE (5-fold): 0.7205

Ridge coefficients:
Latitude      -0.896853
Longitude     -0.869765
MedInc        0.854377
AveBedrms     0.339234
AveRooms      -0.294390
HouseAge      0.122554
AveOccup      -0.040829
Population    -0.002305
dtype: float64

```

```

# ===== 8) Random Forest Regressor (GridSearchCV) =====
rf = RandomForestRegressor(random_state=RANDOM_STATE)
param_grid_rf = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20]
}

```

```
gs_rf = GridSearchCV(rf, param_grid_rf, scoring='neg_mean_squared_error', cv=3,
n_jobs=-1)
```

```
gs_rf.fit(X_train, y_train) # RandomForest works fine on unscaled data; we pass
original X_train
```

```
best_rf = gs_rf.best_estimator_
```

```
print("Random Forest best params:", gs_rf.best_params_)
```

```
# Evaluate RF on the same scaled test input shape: we must pass unscaled X_test
because RF was trained on unscaled X_train
```

```
res_rf = evaluate_model("RandomForest (best)", best_rf, X_train.values,
X_test.values, y_train, y_test)
```

```
Random Forest best params: {'max_depth': None, 'n_estimators': 200}
C:\Users\sarve\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py
:2749: UserWarning: X does not have valid feature names, but RandomForestRegressor was fi
tted with feature names
  warnings.warn(
C:\Users\sarve\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py
:2749: UserWarning: X does not have valid feature names, but RandomForestRegressor was fi
tted with feature names
  warnings.warn(
---- RandomForest (best) ----
MAE: 0.3268
MSE: 0.2540
RMSE: 0.5040
R2: 0.8062
AdjR2:0.8058
Train CV RMSE (5-fold): 0.5106
```

```
# Feature importances from Random Forest
```

```
feat_imp = pd.Series(best_rf.feature_importances_,
index=X.columns).sort_values(ascending=False)
```

```
print("Random Forest feature importances:\n", feat_imp)
```

```
print()
```

```
Random Forest feature importances:
MedInc      0.525886
AveOccup    0.138055
Latitude    0.088647
Longitude    0.088307
HouseAge    0.054355
AveRooms    0.044449
Population  0.030693
AveBedrms   0.029608
dtype: float64
```

```
# ===== 9) Compare results in a DataFrame =====
```

```
results = pd.DataFrame([
    {'model': res_lin['name'], 'rmse': res_lin['rmse'], 'r2': res_lin['r2'], 'adj_r2':
res_lin['adj_r2']},
    {'model': res_ridge['name'], 'rmse': res_ridge['rmse'], 'r2': res_ridge['r2'], 'adj_r2':
res_ridge['adj_r2']},
    {'model': res_rf['name'], 'rmse': res_rf['rmse'], 'r2': res_rf['r2'], 'adj_r2': res_rf['adj_r2']}
])
print("Summary comparison:\n", results.sort_values('rmse'))
print()
```

```
Summary comparison:
   model      rmse      r2      adj_r2
0  RandomForest (best)  0.503960  0.806186  0.805809
1  RandomForest (best)  0.503960  0.806186  0.805809
2      Ridge (best)  0.745579  0.575791  0.574967
3  LinearRegression  0.745581  0.575788  0.574964
```

```
# ===== 10) Choose best model (lowest RMSE on test) =====
```

```
best_idx = results['rmse'].idxmin()
best_model_name = results.loc[best_idx, 'model']
print("Best model on test RMSE:", best_model_name)
```

```
Best model on test RMSE: RandomForest (best)
2  RandomForest (best)  0.503960  0.806186  0.805809
1      Ridge (best)  0.745579  0.575791  0.574967
0  LinearRegression  0.745581  0.575788  0.574964
```

```
# Map name to estimator and the appropriate scaler/feature usage
```

```
if best_model_name.startswith("RandomForest"):
```

```

best_model = best_rf
X_for_plot_test = X_test.values
y_pred = res_rf['y_pred']
elif best_model_name.startswith("Ridge"):
    best_model = best_ridge
    X_for_plot_test = X_test_scaled
    y_pred = res_ridge['y_pred']
else:
    best_model = lin
    X_for_plot_test = X_test_scaled
    y_pred = res_lin['y_pred']

# ===== 11) Diagnostic plots for the chosen model =====
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.scatter(y_test, y_pred, alpha=0.6)
minv = min(y_test.min(), np.min(y_pred))
maxv = max(y_test.max(), np.max(y_pred))
plt.plot([minv, maxv], [minv, maxv], '--', linewidth=1, label='Ideal')
plt.xlabel("Actual median value")
plt.ylabel("Predicted median value")
plt.title(f"Actual vs Predicted ({best_model_name})")
plt.legend()
plt.grid(alpha=0.3)

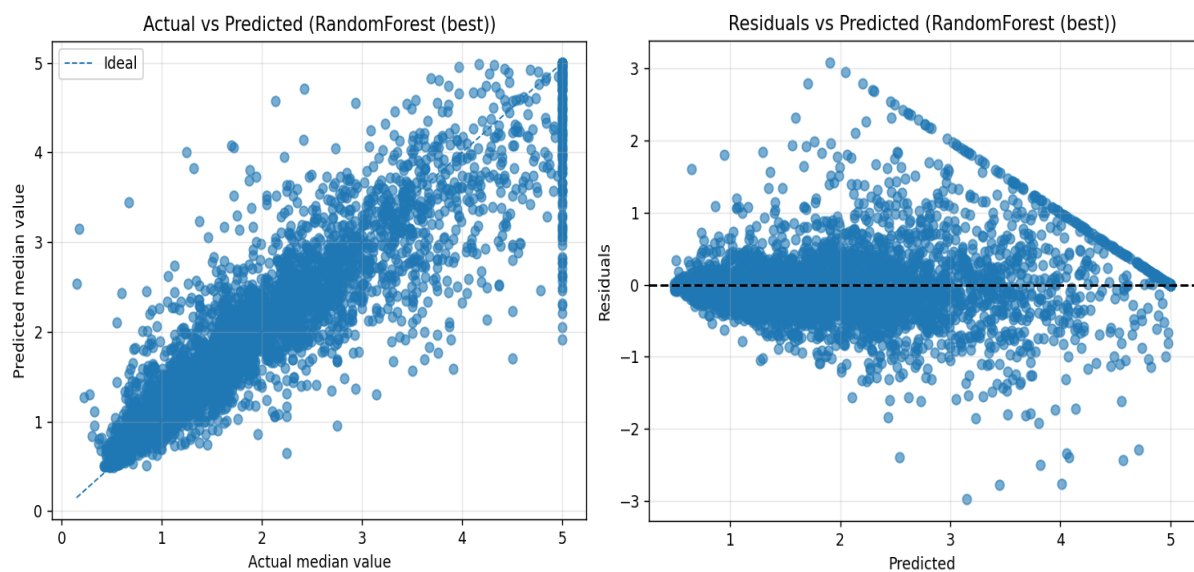
plt.subplot(1,2,2)
residuals = y_test.values - y_pred
plt.scatter(y_pred, residuals, alpha=0.6)

```

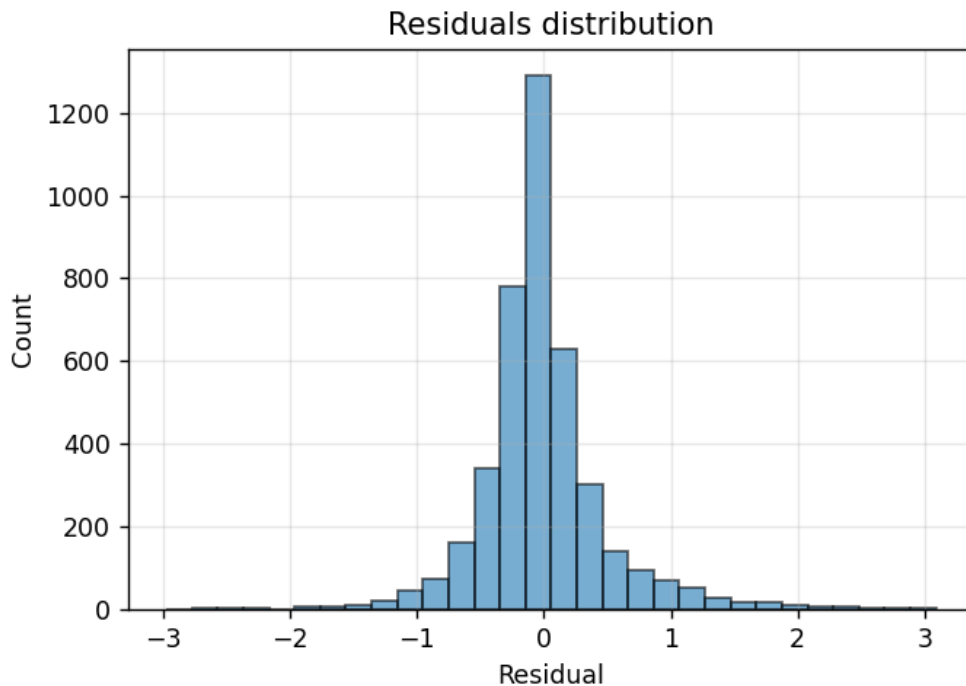
```
plt.axhline(0, linestyle='--', color='black')
plt.xlabel("Predicted")
plt.ylabel("Residuals")
plt.title(f"Residuals vs Predicted ({best_model_name})")
plt.grid(alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
plt.figure(figsize=(6,4))
plt.hist(residuals, bins=30, edgecolor='k', alpha=0.6)
plt.title("Residuals distribution")
plt.xlabel("Residual")
plt.ylabel("Count")
plt.grid(alpha=0.3)
plt.show()
```

===== 12) Save the chosen model (and scaler if used) =====

```
save_name = "best_house_price_model.joblib"
```

```
if best_model_name.startswith("RandomForest"):
```

```
    # RF trained on unscaled data
```

```
    joblib.dump({'model': best_model, 'scaler': None, 'features': list(X.columns)},
                save_name)
```

```
else:
```

```
    joblib.dump({'model': best_model, 'scaler': scaler, 'features': list(X.columns)},
                save_name)
```

```
print(f"Saved best model to {save_name}")
```

Saved best model to best_house_price_model.joblib

===== 13) Example predictions on new samples =====

```
print("\nExample predictions on first 5 test rows:")
```

```
sample_X = X_test.iloc[:5]
```

```
print("Input (first 5 rows):\n", sample_X)
```

```
if best_model_name.startswith("RandomForest"):
```

```
    preds = best_model.predict(sample_X.values)
```

else:

```
preds = best_model.predict(scaler.transform(sample_X))  
print("Predicted median house values:", np.round(preds, 3))  
print("Actual:", np.round(y_test.iloc[:5].values, 3))
```

```
Example predictions on first 5 test rows:  
Input (first 5 rows):  
      MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  Longitude  
20046  1.6812    25.0  4.192201  1.022284    1392.0  3.877437    36.06    -119.01  
3024   2.5313    30.0  5.039384  1.193493    1565.0  2.679795    35.14    -119.46  
15663  3.4801    52.0  3.977155  1.185877    1310.0  1.360332    37.80    -122.44  
20484  5.7376    17.0  6.163636  1.020202    1705.0  3.444444    34.28    -118.72  
9814   3.7250    34.0  5.492991  1.028037    1063.0  2.483645    36.62    -121.93  
C:\Users\sarve\AppData\Roaming\Python\Python312\site-packages\sklearn\utils\validation.py:2749:  
UserWarning: X does not have valid feature names, but RandomForestRegressor was fitted with fe  
ature names  
warnings.warn(  
Predicted median house values: [0.494 0.736 4.877 2.55  2.258]  
Actual: [0.477 0.458 5.    2.186 2.78 ]
```

===== 14) Short conclusion =====

```
print("\nConclusion:")  
print("- LinearRegression gives a quick baseline.")  
print("- Ridge reduced overfitting in coefficients (if observed).")  
print("- Random Forest usually yields better test RMSE/R2 due to capturing  
nonlinearity & interactions.")  
print("- Use feature importances and diagnostic plots to interpret and validate the  
model.")  
print("- For production: consider more thorough hyperparameter tuning, ensemble  
methods, and additional feature engineering (spatial features, interactions).")
```

```
Conclusion:  
- LinearRegression gives a quick baseline.  
- Ridge reduced overfitting in coefficients (if observed).  
- Random Forest usually yields better test RMSE/R2 due to capturing nonlinearity & interactions  
.  
- Use feature importances and diagnostic plots to interpret and validate the model.  
- For production: consider more thorough hyperparameter tuning, ensemble methods, and additional  
feature engineering (spatial features, interactions).
```

EXPERIMENT NO.: 10		
Title of Experiment: Implement and demonstrate the FIND-S (EM algorithm) algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.		
Program: B.Tech CSE	Course: Machine Learning LAB Course Code: PCS 311(L)	Semester: V
Page no: ¼		

1. Aim

Find the **most specific hypothesis** consistent with all **positive** training examples, using the FIND-S algorithm. Read training examples from a CSV file.

2. Theory (short)

- Hypothesis H is represented as a tuple/list of attribute constraints, one constraint per attribute.
- We represent the most specific hypothesis as $\emptyset$ (in code: `None`) — meaning “no value yet allowed”.
- The general hypothesis value $?$ means “any value is allowed” (wildcard).
- Algorithm: start with most specific $h = [\emptyset, \emptyset, \dots]$. For each **positive** training example, generalize constraints in h only as much as needed to be consistent with that example. (Ignore negative examples.)

Pseudo-code

Initialize h to most specific (all $\emptyset$)

For each training example x :

if x is positive:

for each attribute i :

if $h[i] == \emptyset$:

$h[i] = x[i]$

elif $h[i] != x[i]$:

$h[i] = '?'$

else:

h[i] equals x[i] -> do nothing

Output hypothesis h

3. CSV input format (expected)

- First row: header with attribute names.
- Last column: class label (e.g. `PlayTennis`), containing two values (positive/negative).
- Example header: `Outlook, Temperature, Humidity, Wind, PlayTennis`
- Example rows: `Sunny, Hot, High, Weak, No` etc.

(If your CSV uses a different label column or different positive label (e.g. `Yes, Positive, 1`), the program below lets you specify those.)

4. Implementation choices / notes

- Hypothesis representation: Python list where each position corresponds to an attribute.
 - `None` → most specific ($\emptyset$)
 - concrete value → specific constraint
 - `'?'` → wildcard (most general)
- Comparisons are done by equality (works for categorical attributes). Numeric attributes can be used but are matched by exact equality (if you want range support, we can add it).
- The program prints **stepwise updates** whenever a positive example causes a change.

5. Demonstration (short walkthrough)

Using the classic `PlayTennis` example (CSV provided below), the program prints initial hypothesis and its update after each positive example. Final hypothesis will be printed and you can optionally evaluate the hypothesis against the dataset (predict positive only if all attribute constraints match).

Python Code

- read a CSV you supply (or create and use a sample `play_tennis.csv` if none provided),
- run FIND-S showing step updates,
- output final hypothesis,
- optionally save predictions to a CSV.

```
import pandas as pd
```

```
import argparse
```

```
import os
```

```
import sys
```

```
def find_s(df, target_col, positive_label=None, verbose=True):
```

```
    """
```

```
    Run FIND-S on pandas DataFrame df.
```

```
    - target_col: name of the class/label column
```

```
    - positive_label: value in target_col treated as positive (case-insensitive). If None,  
    auto-detect common labels.
```

```
    Returns: (features, final_hypothesis_list, positive_label_used, steps_list)
```

```
    where steps_list is a list of tuples (step_description, hypothesis_snapshot)
```

```
    """
```

```
    df = df.copy()
```

```
    # Heuristic detection of positive label if not provided
```

```
    if positive_label is None:
```

```
        unique = df[target_col].unique().tolist()
```

```
        candidates = [u for u in unique if str(u).strip().lower() in  
        ('yes','y','positive','1','true','t')]
```

```
        positive_label = candidates[0] if candidates else unique[0]
```

```
    features = [c for c in df.columns if c != target_col]
```

```
    h = [None] * len(features) # None means  $\emptyset$  (most specific)
```

```
    steps = []
```

```
    steps.append(("Initial", h.copy()))
```

```
    if verbose:
```

```

print("Initial hypothesis:", [ "Ø" for _ in features ])

for idx, row in df.iterrows():
    row_label = str(row[target_col]).strip().lower()
    if row_label == str(positive_label).strip().lower():
        # positive example -> generalize h as needed
        for i, feat in enumerate(features):
            value = row[feat]
            if h[i] is None:
                # most specific -> set to this value
                h[i] = value
            elif h[i] != value:
                # conflict -> generalize to '?'
                h[i] = '?'
            else:
                # h[i] == value -> no change
                pass
        steps.append((f"After positive example row {idx+1}", h.copy()))
    if verbose:
        readable = [ (h_i if h_i is not None else 'Ø') for h_i in h ]
        print(f"After positive example {idx+1}: {[row[f] for f in features]} -> hypothesis: {readable}")

```

Output -

```

CSV file 'play_tennis.csv' not found. Creating a sample PlayTennis dataset at that path for demonstration.
Sample dataset written to: play_tennis.csv
No target column specified; using last column: 'PlayTennis '
Initial hypothesis: ['Ø', 'Ø', 'Ø', 'Ø']
After positive example 3: ['Overcast', 'Hot', 'High', 'Weak'] -> hypothesis: ['Overcast', 'Hot', 'High', 'Weak']
After positive example 4: ['Rain', 'Mild', 'High', 'Weak'] -> hypothesis: ['?', '?', 'High', 'Weak']
After positive example 5: ['Rain', 'Cool', 'Normal', 'Weak'] -> hypothesis: ['?', '?', '?', 'Weak']
After positive example 7: ['Overcast', 'Cool', 'Normal', 'Strong'] -> hypothesis: ['?', '?', '?', '?']
After positive example 9: ['Sunny', 'Cool', 'Normal', 'Weak'] -> hypothesis: ['?', '?', '?', '?']
After positive example 10: ['Rain', 'Mild', 'Normal', 'Weak'] -> hypothesis: ['?', '?', '?', '?']
After positive example 11: ['Sunny', 'Mild', 'Normal', 'Strong'] -> hypothesis: ['?', '?', '?', '?']
After positive example 12: ['Overcast', 'Mild', 'High', 'Strong'] -> hypothesis: ['?', '?', '?', '?']
After positive example 13: ['Overcast', 'Hot', 'Normal', 'Weak'] -> hypothesis: ['?', '?', '?', '?']

```

```
return features, h, positive_label, steps
```

```
def predict_with_h(h, features, instance):
```

```
    """
```

Return True if hypothesis h classifies instance (pandas Series or dict) as positive.

Matching rules:

- h[i] == '?' matches any value

- h[i] == None (∅) matches nothing (i.e., hypothesis too specific -> does not match)

- otherwise require equality

```
    """
```

```
    for i, feat in enumerate(features):
```

```
        constraint = h[i]
```

```
        val = instance[feat]
```

```
        if constraint == '?':
```

```
            continue
```

```
        if constraint is None:
```

```
            return False
```

```
        if constraint != val:
```

```
            return False
```

```
    return True
```

```
def evaluate_hypothesis(h, features, df, target_col, positive_label=None):
```

```
    """
```

Apply hypothesis on dataset and return (preds, accuracy, TP, TN, FP, FN)

```
    """
```

```
    if positive_label is None:
```

```
        unique = df[target_col].unique().tolist()
```

```
        candidates = [u for u in unique if str(u).strip().lower() in  
('yes','y','positive','1','true','t')]
```

```

    positive_label = candidates[0] if candidates else unique[0]
preds = []
TP = TN = FP = FN = 0
for idx, row in df.iterrows():
    pred_pos = predict_with_h(h, features, row)
    actual_pos = (str(row[target_col]).strip().lower() ==
str(positive_label).strip().lower())
    preds.append('Yes' if pred_pos else 'No')
    if pred_pos and actual_pos:
        TP += 1
    elif pred_pos and not actual_pos:
        FP += 1
    elif not pred_pos and actual_pos:
        FN += 1
    else:
        TN += 1
total = len(df)
accuracy = (TP + TN) / total if total > 0 else 0.0
return preds, accuracy, TP, TN, FP, FN

```

```

def create_sample_csv(path='play_tennis.csv'):
    sample = """Outlook,Temperature,Humidity,Wind,PlayTennis
Sunny,Hot,High,Weak,No
Sunny,Hot,High,Strong,No
Overcast,Hot,High,Weak,Yes
Rain,Mild,High,Weak,Yes
Rain,Cool,Normal,Weak,Yes
Rain,Cool,Normal,Strong,No

```


Overcast,Cool,Normal,Strong,Yes

Sunny,Mild,High,Weak,No

Sunny,Cool,Normal,Weak,Yes

Rain,Mild,Normal,Weak,Yes

Sunny,Mild,Normal,Strong,Yes

Overcast,Mild,High,Strong,Yes

Overcast,Hot,Normal,Weak,Yes

Rain,Mild,High,Strong,No

"""

with open(path, 'w', newline='') as f:

 f.write(sample)

print(f"Sample dataset written to: {path}")



```
play_tennis.csv X
Python > machine_learning > play_tennis.csv > data
1 Outlook, Temperature, Humidity, Wind, PlayTennis
2 Sunny, Hot, High, Weak, No
3 Sunny, Hot, High, Strong, No
4 Overcast, Hot, High, Weak, Yes
5 Rain, Mild, High, Weak, Yes
6 Rain, Cool, Normal, Weak, Yes
7 Rain, Cool, Normal, Strong, No
8 Overcast, Cool, Normal, Strong, Yes
9 Sunny, Mild, High, Weak, No
10 Sunny, Cool, Normal, Weak, Yes
11 Rain, Mild, Normal, Weak, Yes
12 Sunny, Mild, Normal, Strong, Yes
13 Overcast, Mild, High, Strong, Yes
14 Overcast, Hot, Normal, Weak, Yes
15 Rain, Mild, High, Strong, No
```

def main(args):

 # load CSV (or create sample if not found)

```

if args.csv is None:
    csv_path = "play_tennis.csv"
else:
    csv_path = args.csv

if not os.path.exists(csv_path):
    print(f"CSV file '{csv_path}' not found. Creating a sample PlayTennis dataset at
that path for demonstration.")
    create_sample_csv(csv_path)

try:
    df = pd.read_csv(csv_path)
except Exception as e:
    print("Failed to read CSV:", e)
    sys.exit(1)

# if label not specified, default to last column
if args.target is None:
    target_col = df.columns[-1]
    print(f"No target column specified; using last column: '{target_col}'")
else:
    if args.target not in df.columns:
        print(f"Target column '{args.target}' not found in CSV columns:
{list(df.columns)}")
        sys.exit(1)
    target_col = args.target

# Run FIND-S

```

```
features, h, pos_label, steps = find_s(df, target_col, positive_label=args.positive, verbose=True)
```

```
print("\nFinal hypothesis (feature -> constraint):")
```

```
readable = [ (h_i if h_i is not None else 'Ø') for h_i in h ]
```

```
for feat, constraint in zip(features, readable):
```

```
    print(f" {feat} -> {constraint}")
```

```
Final hypothesis (feature -> constraint):
Outlook  -> ?
Temperature -> ?
Humidity  -> ?
Wind      -> ?
```

```
# Evaluate hypothesis on same dataset (informative only)
```

```
preds, acc, TP, TN, FP, FN = evaluate_hypothesis(h, features, df, target_col, pos_label)
```

```
print("\nEvaluation on dataset (hypothesis predictions):")
```

```
print(f" Positive label used: {pos_label}")
```

```
print(f" Accuracy: {acc:.3f} (TP={TP}, TN={TN}, FP={FP}, FN={FN})")
```

```
# Optionally save predictions to csv
```

```
if args.output is not None:
```

```
    out_df = df.copy()
```

```
    out_df['FIND_S_Pred'] = preds
```

```
    out_df.to_csv(args.output, index=False)
```

```
    print(f"Predictions saved to: {args.output}")
```

Output -

```
Evaluation on dataset (hypothesis predictions):
Positive label used: Yes
Accuracy: 0.643 (TP=9, TN=0, FP=5, FN=0)
```

```
if __name__ == "__main__":  
    parser = argparse.ArgumentParser(description="FIND-S algorithm on CSV  
dataset")  
  
    parser.add_argument('--csv', type=str, help='Path to input CSV (default:  
play_tennis.csv created for demo)')  
  
    parser.add_argument('--target', type=str, help='Name of target/class column  
(default: last column)')  
  
    parser.add_argument('--positive', type=str, help='Value in target column to treat as  
positive (case-insensitive). If omitted, auto-detected.')  
  
    parser.add_argument('--output', type=str, help='Optional output CSV to write  
predictions into')  
  
    args = parser.parse_args()  
  
    main(args)
```